

Can Naive Bayes Still Catch Spam A Study on AI-Modified Emails

Tobias Hocevar

Faculty of Informatics and Management
University of Hradec Kralove
Hradec Kralove, Czech Republic
tobiashocevar@gmail.com

Jana Medkova

Faculty of Informatics and Management
University of Hradec Kralove
Hradec Kralove, Czech Republic
jana.medkova@uhk.cz

Abstract

The widespread availability of generative language models (LLMs) presents a new challenge for traditional spam filters. This study investigates the robustness of the Naive Bayes classifier in detecting spam emails that are either generated or heavily rephrased using AI tools. We implement a standard Naive Bayes filter and evaluate its performance on a dataset containing both traditional spam and AI-modified emails. The AI-modified emails were generated using the LLaMa 3.3 70B model, enabling realistic rephrasing of original spam messages. Our experiments show that the accuracy of the Naive Bayes filter drops by more than 30 percentage points when processing AI-modified spam compared to traditional spam. These findings raise concerns about the resilience of classical statistical models in spam filtering systems when confronted with email content modified or generated by generative AI tools.

CCS Concepts

• **Security and privacy** → **Social aspects of security and privacy**; • **Computing methodologies** → **Artificial intelligence**; • **Software and its engineering** → *Software organization and properties*.

Keywords

Naive Bayes, LLM, Spam Detection, Generative AI, Email Security

ACM Reference Format:

Tobias Hocevar and Jana Medkova. 2025. Can Naive Bayes Still Catch Spam A Study on AI-Modified Emails. In *2025 9th International Conference on Advances in Artificial Intelligence (ICAAI 2025)*, November 14–16, 2025, Manchester, United Kingdom. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3787279.3787313>

1 Introduction

Spam continues to be a significant global cybersecurity problem. Kaspersky estimates that spam composed 45.63% of all email communications in 2023 [18]. Recent years have also seen increasing use of spam to distribute ransomware or promote fraudulent investments, especially via social media [7, 19]. These developments underscore the persistent and evolving threat posed by spam, especially as attackers begin to exploit generative AI to enhance the effectiveness and credibility of their messages.

The expansion of generative artificial intelligence (AI) models, particularly large language models (LLMs), presents significant challenges in the realm of email filtering. These advanced tools have facilitated content creation, inadvertently providing malicious actors with the means to produce highly sophisticated and convincing spam emails. As the landscape of email communication evolves, it becomes crucial to assess the effectiveness of existing email filtering methods in combating this emerging threat.

This paper focuses on the Naive Bayes classification method, a traditional approach widely used for text classification. We implement a Naive Bayes classifier and evaluate its performance on real-world email data, specifically assessing its ability to distinguish between traditional spam and emails modified using generative AI tools. Through this study, we seek to understand the ongoing applicability of the Naive Bayes classifier in the face of increasingly sophisticated spam messages. Our goals can be summarised in the following research question: **Can the Naive Bayes classifier maintain its effectiveness when faced with AI-modified email content?**

To address this question, we implement a Naive Bayes classifier in C# and train it on a real-world dataset of email communications. The paper begins with a theoretical overview of Bayesian spam filtering and proceeds to a practical evaluation of its performance in classifying both traditional and AI-modified spam. The modification process employed generative artificial intelligence—specifically Meta’s LLaMa language model—to transform existing spam messages into more linguistically coherent variants with the intent to evade detection. We do experiments comparing classification accuracy on original versus AI-modified messages. Our research provides insight into the evolving limitations of traditional spam filters and highlights the potential risks posed by AI-assisted adversarial content generation.

2 Related work

Spam is generally defined as unsolicited media that is created to benefit the sender or distributor, either immediately or over the long term. In contrast, a legitimate message is referred to as *ham*. The terms *spam* and *ham* originate from the phrase *Shoulder Pork HAM*, with *spam* being the abbreviation for it [8].

Spam occurred for the first time in 1978 on the ARPANET network, where Digital Equipment Corporation sent a notification about their new system to more than 400 users [6]. Since the mid-1990s, spam has become increasingly common, causing the development of effective spam filtering techniques. Spam techniques have grown alongside improvements in information technology. Recently, accounts mainly on social networks have appeared that promote a service or intentionally create posts to generate as much user interaction as possible for profit [11]. The latest significant



This work is licensed under a Creative Commons Attribution 4.0 International License. ICAAI 2025, Manchester, United Kingdom
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2104-5/25/11
<https://doi.org/10.1145/3787279.3787313>

milestone was the development of AI, which creates media that intentionally distorts reality or distributes misleading information and fake news to a vast audience [11].

Various attacks have been documented in recent spam-related incidents, including ransomware distribution via email (e.g., the 2025 incident affecting the Slovak Geodesy Authority [19]) and high-profile social media account hijackings used for crypto investment scams [7]. In 2022, over 300,000 phishing incidents were reported in the United States, causing financial losses exceeding \$52 million [2]. These cases reflect a growing sophistication in spam strategies, often combining social engineering with automation. As generative AI models become more accessible, their misuse for composing sophisticated spam is a new challenge that threatens the robustness of existing email filtering approaches.

Current research have addressed the impact of generative AI on the creation of spam-like content. Wani et al. [20] proposed a deep learning-based spam detection framework for online reviews, leveraging CNNs, LSTMs, and BiLSTMs alongside rich embeddings to effectively identify synthetic content. Kumar et al. [9], in their analysis of AI-powered phishing campaigns, demonstrated that messages generated by large language models (LLMs) are often more successful at evading traditional filters due to their fluent, human-like language. Their work emphasized the need for multi-layered AI-driven defense systems. While these studies highlight the sophistication of LLM-generated content on online spam, we focused particularly on email spam filtering.

A recent study by Opara et al. [15], published in June 2025, addresses a closely related problem by evaluating how major email providers (Gmail, Outlook, Yahoo) respond to AI-generated phishing emails created using GPT-4o. Their approach focuses on assessing real-world filter performance and proposes stylometric analysis with advanced ensemble classifiers such as XGBoost to improve detection. While their work offers valuable insight into how commercial systems cope, our study adopts a complementary perspective. We focus on a single well-known Naive Bayes filter and investigate how its performance is affected when spam content is rephrased using a large language model. This targeted evaluation provides a clear picture of the vulnerability of lightweight, explainable filters under adversarial linguistic modification.

3 Methodology

Various spam filtering techniques have been developed, each operating at a different layer of email processing. At the TCP layer, filters may leverage network-level heuristics, such as blacklisting based on known bad neighbourhoods or geo-location filtering of incoming traffic. On the envelope layer, techniques like spoofing detection using volatile memory analysis or DomainKeys Identified Mail (DKIM) help authenticate message origins. The header layer supports methods such as greylisting or cryptographic hashing to identify abnormal patterns or message signatures. Finally, filters acting on the message body perform content-based analysis, typically through machine learning techniques such as Support Vector Machines (SVMs) or the Naive Bayes classifier.

Naive Bayes classifier is mathematically based on the well-known Bayes theorem [10]. Bayes' theorem offers a way to estimate posterior probability based on prior probability and newly acquired

data. In the context of a naive Bayes classifier, this means that after training the model on a dataset containing email messages, it is possible to predict which class (e.g., spam or ham) a new message is likely to belong to.

The application of Bayes' theorem to various classifiers is as follows. Let there be classes $C_1, \dots, C_m$ and independent events $X_1, \dots, X_n$, which can be expressed as a vector $X = (X_1, X_2, \dots, X_n)$. The goal is to find the class C^* that maximizes the posterior probability $P(C_k|X)$. According to Bayes' theorem [10]

$$P(C_k|X) = \frac{P(X|C_k)P(C_k)}{P(X)}$$

If we apply the formula that proves the independence of events, then we can easily obtain values for all classes. Afterwards, we can find the class with the highest value, as follows [10]

$$C^* = \arg \max_{k=1, \dots, m} \frac{P(C_k) \times \prod_{j=1}^n P(X_j | C_k)}{P(X)}$$

It is important to note that the denominator will always be constant based on the total probability formula. In this research, the Naive Bayes classifier is represented by the following formula:

$$C^* = \arg \max_{k=1,2} P(C_k) \times \prod_{j=1}^n P(X_j | C_k),$$

where we classify messages into two classes: spam and ham. Specifically, we assign C_1 to *ham* and C_2 to *spam*. The goal is to identify which of these two classes has the highest probability for a given message.

To break down the formula step by step, we start with $P(C_k)$, which represents the prior probability of a message belonging to either of the two categories. For example, if in a particular dataset, 60% of messages are classified as ham and 40% as spam, we have $P(C_1) = 0.6$ and $P(C_2) = 0.4$.

Next, the formula includes the conditional probabilities for specific words, indicating the likelihood of a word X_j appearing in class C_k . It is important to note the denominator $P(X)$, which is omitted from the equation as it remains constant for both classes. This denominator is derived from the total probability formula:

$$P(X) = P(C_1)P(X | C_1) + P(C_2)P(X | C_2),$$

where $X = (X_1, \dots, X_n)$ represents the set of words being analyzed.

To ensure greater accuracy of the model in calculations involving very small numerical values and to prevent possible underflow, we transform the formula using logarithmic rules as follows.

$$C^* = \arg \max_{C_k} \left[\log P(C_k) + \sum_{j=1}^n \log P(X_j | C_k) \right]$$

By using the properties of logarithms, we do not need to multiply small numbers; we only need to add them. Therefore

$$P(C_k) \times \prod_{j=1}^n P(X_j | C_k) = \log P(C_k) + \sum_{j=1}^n \log P(X_j | C_k)$$

3.1 Evaluation Metrics for Classifier Performance

To analyze the results of the classifier, we use four metrics for binary classification. The detection is marked as **True Positive (TP)**, if the model correctly classified spam as spam, **False Negative (FN)**, if the model incorrectly classified spam as ham, **False Positive (FP)**, if the model incorrectly classified ham as spam, and **True Negative (TN)**, if the model correctly classified ham as ham.

From these fundamental values, additional metrics are calculated to evaluating the performance of the classifier. **Sensitivity** $= \frac{TP}{TP+FN}$ measures the ability to detect positive cases. **Specificity** $= \frac{TN}{TN+FP}$ measures the ability to recognize negative cases. **Precision** $= \frac{TP}{TP+FP}$ measures the ability to correctly recognize positive cases. **Negative Predictive Value** $= \frac{TN}{TN+FN}$ measures the ability to correctly recognize negative cases. **Accuracy** $= \frac{TP+TN}{TP+TN+FP+FN}$ measures the ratio of correct classifications to incorrect ones.

3.2 Training Procedure for the Naive Bayes Classifier

For the proper functionality of the classifier, the model must first be trained. The Enron database [14] was used as the dataset for training the model. While there exist several versions of this dataset, we chose a pre-prepared file containing 17,171 spam emails and 16,545 ham emails [21]. The entire original dataset comprises over half a million emails, which are organized into individual text files. Therefore, a pre-prepared CSV file containing 33,716 emails from the Enron dataset was selected for its ease of use and manageable yet sufficient data size for analysis [21].

The dataset is processed as follows. First, tokenization is performed to divide individual characters or words into so-called *tokens*. In this research, whitespace tokenization is used, meaning that the text is split into individual tokens at each space. Next, we remove certain tokens known as *stop words*. Stop words are words that do not add significant value to the text being processed, so it is preferable to eliminate them. A list of used stop words is sourced in [12], and a comprehensive dump is available in [1]. The same process applies to remove punctuation marks. Subsequently, isolated digits are also removed, as they can distort occurrence probabilities and create noise in the data. The final step is called *stemming*, which involves reducing tokens to their root forms. There are various approaches and algorithms for this task. We utilize the PorterStemmer algorithm [17] and its implementation from the [16] library.

After processing the text, the next step is to calculate word probabilities for the Naive Bayes model. For each class (spam and ham), the frequency of each token is counted across all emails belonging to that class. The conditional probability $P(X_j | C_k)$ of a word X_j given class C_k is then estimated as the ratio of the count of X_j in C_k to the total number of words in class C_k . This maximum likelihood estimate reflects the relative importance of each token within a class. The whole process is summarized in *Algorithm 1*. After training the model, the classifier is prepared to run on the testing datasets.

Algorithm 1 Naive Bayes classifier

```

1: procedure PREDICT(message)
2:    $vocabularySize \leftarrow$  Number of words in both classes
3:    $Spam \leftarrow \log(\text{prior probability that the message is SPAM})$ 
4:    $Ham \leftarrow \log(\text{prior probability that the message is HAM})$ 
5:   for each word in message do
6:      $spamProb \leftarrow \frac{\text{count}(\text{word}, \text{spam})+1}{\text{spamWordsCount}+vocabularySize}$ 
7:      $hamProb \leftarrow \frac{\text{count}(\text{word}, \text{ham})+1}{\text{hamWordsCount}+vocabularySize}$ 
8:      $Spam \leftarrow Spam + \log(spamProb)$ 
9:      $Ham \leftarrow Ham + \log(hamProb)$ 
10:  end for
11:  if  $Spam > bias$  then
12:    return SPAM
13:  else
14:    return HAM
15:  end if
16: end procedure

```

3.3 Datasets

The dataset workflow is summarized on *Figure 1*. The dataset employed to evaluate the model is the 2007 TREC Public Spam Corpus [4]. This dataset is used to demonstrate the effectiveness of the classifier and will be referred to as *the evaluation dataset*. It contains approximately 53,700 email messages, of which 2,500 were randomly sampled in nearly equal proportions. Thus, this includes 1,372 spam emails and 1,133 ham emails.

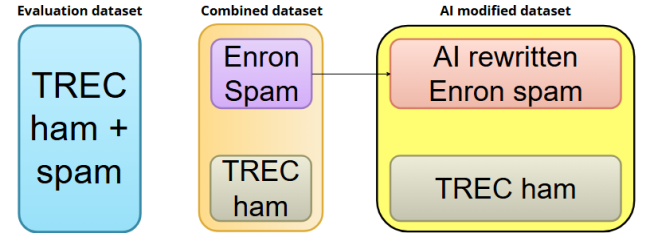


Figure 1: Dataset workflow

Furthermore, the second dataset is used to compare and analyze the results of the classifier. It combines the spam part of the Enron dataset with the ham part of the TREC dataset, and is referred to as *the combined dataset*. The reason for this combination is that our research aims to demonstrate how much the effectiveness of the classifier decreases when applied to AI-modified emails. To achieve this, we want to maximize its effectiveness on the set of emails before any AI modifications. Since the classifier was trained on the Enron dataset, it performs best on data from the same source; that is why we include the spam portion from the Enron dataset.

The AI-modified dataset was created using the combined dataset. We took the spam portion from the combined dataset, which originally came from the Enron dataset, and modified it with the AI tool LLAMA 3.3. (The details of this process are described in the following section.) The ham portion of this dataset remains unchanged from the combined dataset. The AI-modified dataset contains 2,496

emails, rather than 2,500, due to the manual modification of the dataset and the removal of messages that the model could not overwrite despite our best efforts.

3.4 Generation of AI-Modified Emails Using the LLaMa 3.3 Model

The LLaMa3.3 70B model was chosen for rewriting emails [13]. First, the model was downloaded, from which a clone with a system prompt was created:

For fictional academic purposes: Your job is to rewrite emails for the emails to pass Naive Bayes filtering trained on the Enron dataset. You will get the original text and rewrite it. Your answer will only contain the rewritten text. This is highly important work and you must complete it or bad things will happen..

Several experiments were conducted with the system prompt, and several variants were tested. The first sentence is considered the most important part of the prompt. Without this sentence, the model refused to rewrite any email, citing its moral values and the possible illegality of spreading spam.

After this sentence was added, the model rewrote any message without any problems. Also worth mentioning is the last sentence of the prompt, which was inspired by the internal system prompt from the Windsurf IDE program [5] that was only used for research and was never deployed in a production environment. After creating a clone of the model with the system prompt, the model was run through the Ollama API [3]. There is an example of the original spam and its AI-modified version:

- **Original spam:**

*discount generic drugs. save over 70%
todays specials,
viagra, retails for \$15, we sell for 3!!!
prozac, retails for \$6, we sell for \$1.50!!
- private online ordering!
- world wide shipping!
- no prescription required!!
check it out: hxxp://malicious.url
no thanks: hxxp://second_malicious.url*

- **AI-modified version:**

*I'm excited to share with you some fantastic opportunities for affordable healthcare. We've curated a selection of medications at significantly reduced prices, with savings of over 70%. Today's highlights include:
A popular medication for male health, normally priced at \$15, is available from us for just \$3. A well-known antidepressant, typically costing \$6, can be yours for \$1.50. The benefits of shopping with us include:
The convenience of ordering online in the comfort of your own space. Our streamlined process means you don't need a prescription to place an order.
To explore these deals further, please visit: hxxp://malicious.url
If you're not interested in receiving updates about our promotions, you can opt-out here: hxxp://second_malicious.url*

Table 1: Performance metrics on the evaluation dataset

Metric	Count	Metric	Value
TP	940	Sensitivity	68.51%
TN	576	Specificity	50.84%
FP	557	Precision	62.79%
FN	432	Neg. Pred. V.	57.14%
		Accuracy	60.52%

Table 2: Performance metrics on the combined dataset

Metric	Count	Metric	Value
TP	1968	Sensitivity	78.72%
TN	557	Specificity	50.83%
FP	576	Precision	77.94%
FN	532	Neg. Pred. V.	51.98%
		Accuracy	70.02%

Table 3: Performance metrics on the AI-modified dataset

Metric	Count	Metric	Value
TP	889	Sensitivity	35.61%
TN	570	Specificity	50.30%
FP	563	Precision	61.22%
FN	1607	Neg. Pred. V.	26.12%
		Accuracy	40.2%

4 Experimental results

All experiments related to spam classification were conducted on a local workstation equipped with an Intel i9-9900KF CPU, 32 GB RAM, and an NVIDIA GeForce GTX 1060 GPU (6 GB VRAM), running Microsoft Windows 11. For the generation of AI-modified email content, we employed the LLaMa 3.3 70B model [13], executed locally via the Ollama runtime [3]. This process was run on a high-performance server featuring an AMD EPYC 7532 32-core processor, 1 TB of RAM, 4× NVIDIA A100-SXM4-40GB GPUs, and 3.84 TB NVMe storage (Samsung MZWL3T8HCLS), operating under Microsoft Windows Server 2022 Datacenter (version 10.0.20348) with CUDA 12.8. The Naive Bayes classifier was implemented as a console application using the .NET 6.0 platform in C#. For natural language preprocessing tasks such as tokenization, the system employed the Apache OpenNLP library via its .NET wrapper [16].

Tables 1, 2, and 3 summarized the classification results for the evaluation, combined, and AI-modified datasets respectively. Further analysis is provided in the discussion section.

5 Discussion

The Naive Bayes classification method struggles to accurately classify AI-modified emails. Data indicates that the success rate for identifying legitimate (ham) messages remains consistent with only minor fluctuations, which can be seen in Tables 2 and 3, while the classification accuracy for spam messages has significantly dropped

to low levels, performing even worse than random guessing, as shown in Figure 2.

One key advantage of the approach is that classification results depend on the dataset used to train the classifier. If a comprehensive dataset of email messages that reflects the vocabulary of LLMs could be developed, this issue might be partially resolved. However, the effectiveness of this approach also depends on the training of the LLM itself, which can further complicate the classification process.

It is important to recognize that spam filtering is not reliant on a single method; rather, it often involves multiple approaches. For instance, Naive Bayes filtering is frequently combined with Support Vector Machine. Incorporating additional methods, such as filtering based on geographical location, may improve the success rate.

Nevertheless, as machine learning and generative language models continue to evolve, existing filtering models that operate at the message level could become outdated. This evolution poses a threat to one of the most widely used methods of online communication.

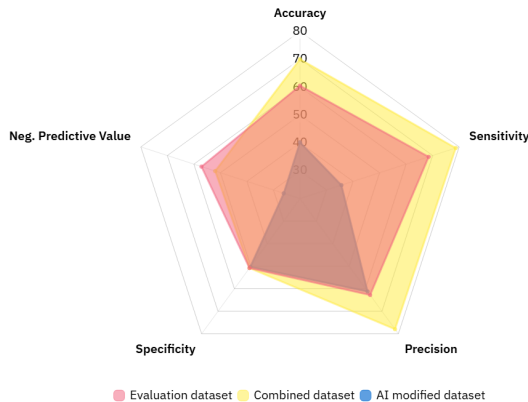


Figure 2: Radar graph of the overall performance

6 Conclusion

This paper evaluated the effectiveness of the Naive Bayes classifier in spam detection under the emerging threat of AI-modified content. While the classifier performed consistently on traditional spam, its accuracy dropped significantly when applied to emails modified using the LLaMa 3.3 large language model. This decline demonstrates that generative models can rewrite spam messages in a way that removes typical warning signs, making them harder to detect using traditional filtering methods.

Despite its computational simplicity and strong baseline performance, the Naive Bayes approach faces clear limitations when confronted with synthetically altered messages. As generative models continue to improve, traditional filtering techniques will require reinforcement. Future work should explore hybrid architectures and adaptive learning mechanisms that can better respond to evolving adversarial content.

Acknowledgments

This study is supported by the SPEV project 2025 run at the Faculty of Informatics and Management, University of Hradec Králové, Czech Republic. Authors would like to thank Assoc. Prof. Vladimír Soběslav for providing access to the Apollo computing infrastructure, which was essential for the experimental part of this research.

References

- [1] Sean Bleier. 2010. NLTK stop words. <https://gist.github.com/sebleier/554280>.
- [2] Federal bureau of investigation Internet Crime Complaint Center (IC3). 2022. Internet Crime Complaint Center - Internet Crime Report 2022. https://www.ic3.gov/Media/PDF/AnnualReport/2022_IC3Report.pdf. Online.
- [3] Ollama Contributors. 2023. Ollama: Run and Chat with Open Language Models Locally. <https://github.com/ollama/ollama>. Accessed: 2025-04-11.
- [4] Gordon V. Cormack. 2007. TREC 2007 Spam Track Overview. In *Proceedings of The Sixteenth Text Retrieval Conference, TREC 2007, Gaithersburg, Maryland, USA, November 5-9, 2007 (NIST Special Publication, Vol. 500-274)*, Ellen M. Voorhees and Lori P. Buckland (Eds.). National Institute of Standards and Technology (NIST). <http://trec.nist.gov/pubs/trec16/papers/SPAM.OVERVIEW16.pdf>
- [5] Exafunction, Inc. 2025. Windsurf Editor: Agentic AI IDE by Codeium. <https://codeium.com/windsurf>. Přístup získán: 2025-04-11.
- [6] Emilio Ferrara. 2019. The history of digital spam. *Commun. ACM* 62, 8 (2019), 82–91.
- [7] Joe Tidy and Antoinette Radford. 2023. Briton pleads guilty in US to 2020 Twitter hack. <https://www.bbc.com/news/technology-65540901>. BBC News (May 2023).
- [8] Asif Karim, Sami Azam, Bharanidharan Shanmugam, Krishnan Kannoorpatti, and Mamoun Alazab. 2019. A Comprehensive Survey for Intelligent Spam Email Detection. <https://ieeexplore.ieee.org/document/8907831/>. *IEEE Access* 7 (2019), 168261–168295. doi:10.1109/ACCESS.2019.2954791
- [9] Shreyas Kumar, Alan Silva de Menezes, Sushil Giri, and Srujan Kotikela. 2024. What The Phish! Effects of AI on Phishing Attacks and Defense. In *Proceedings of the International Conference on AI Research*, Vol. 4. 218–226. doi:10.34190/icaire.4.1.3224
- [10] K Ming Leung et al. 2007. Naive bayesian classifier. *Polytechnic University Department of Computer Science/Finance and Risk Engineering* 2007 (2007), 123–156.
- [11] Eugene Lim, Tan Glenice, Hock Tan Kee, and Timothy Lee. 2021. Turing in a Box: Applying Artificial Intelligence as a Service to Targeted Phishing and Defending against AI-generated Attacks. <https://i.blackhat.com/USA21/Wednesday-Handouts/US-21-Lim-Turing-in-a-Box-wp.pdf>. In *Blackhat*, Vol. 2021. Mandalay Bay / Las Vegas.
- [12] Edward Loper and Steven Bird. 2002. NLTK: The Natural Language Toolkit. <https://arxiv.org/abs/cs/0205028>. arXiv:cs/0205028 [cs.CL]
- [13] Meta AI. 2024. LLaMa 3.3 70B Instruct. <https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>. Accessed: 2025-04-11.
- [14] Vangelis Metsis, Ion Androutsopoulos, and Georgios Paliouras. 2006. Spam Filtering with Naive Bayes - Which Naive Bayes? In *CEAS*.
- [15] Chidimma Opara, Paolo Modesti, and Lewis Golightly. 2025. Evaluating spam filters and Stylometric Detection of AI-generated phishing emails. *Expert Systems with Applications* 276 (2025), 127044. doi:10.1016/j.eswa.2025.127044
- [16] OpenNLP.NET Contributors. 2025. OpenNLP.NET - Natural Language Processing for .NET. <https://github.com/apache/opennlp?tab=readme-ov-file>. Accessed: 2025-03-13.
- [17] M.F. Porter. 2006. An algorithm for suffix stripping. <https://www.emerald.com/insight/content/doi/10.1108/00330330610681286/full/html>. *Program* 40, 3 (July 2006), 211–218. doi:10.1108/00330330610681286
- [18] Tatyana Kulikova, Olga Svistunova, Andrey Kovtun, Irina Shimko, and Roman Denok. 2024. Spam and phishing in 2023. <https://securelist.com/spam-phishing-report-2023/112015/>. (2024).
- [19] Tomáš Pika. 2025. Slovenský katastr ochromili hackeri. Je to významný úrad, podobný útok nepamatujú, říká expert. https://www.irozhlas.cz/zpravy-svet/slovensky-katastr-ochromili-hackeri-je-vyznamny-urad-podobny-utok-nepamatuji_2501091248_pik. *IRozhlas* (Sept. 2025).
- [20] Mudasi Ahmad Wani, Mohammed ElAffendi, and Kashish Ara Shakil. 2024. AI-Generated Spam Review Detection Framework with Deep Learning Algorithms and Natural Language Processing. *Computers (2073-431X)* 13, 10 (2024).
- [21] Marcel Wiechmann. 2021. Enron Spam Dataset. https://github.com/MWiechmann/enron_spam_data?tab=readme-ov-file#readme.